International Conference on Computational Intelligence, Information Technology and Systems Research
June 22–24, 2026, Lublin, Poland
CITS 26

Comparative Analysis of Selected Characteristics of Java and Rust Based on Programs Implementing Equivalent Functionalities

Karol Mac, Łukasz Gałka
Department of Computer Science, Lublin University of Technology
s99058@pollub.edu.pl, l.galka@pollub.pl

Introduction

Choosing the right programming language for implementing a solution to a specific problem is highly important. First, we need to balance application performance and the cost of implementation. Some tasks require fast execution time and limited resources, while others may be implemented with fewer restrictions. In particular, a project often has financial and time constraints that must be fulfilled. On the other hand, from the perspective of software engineering, the codebase should be as easily maintainable as possible. Such limitations have a great impact on programming language selection. Therefore, in this study, we perform a comprehensive analysis and comparison of two popular programming languages. Java represents a high level of language abstraction, while Rust, in contrast,constitutes a low-level programming language.

Research Scenarios

1. ***CPU*** *(Mandelbrot set, 4096×4096, raw PPM output): floating-point-intensive computation with negligible I/O in the hot path; probes the quality of code emitted by the JIT compiler relative to the AOT LLVM backend.*
2. ***I/O*** *(CSV processing, 5GB, 6 × 10^7 rows): two-pass top-1 % transaction-amount filter; evaluates textual parsing throughput, streaming I/O efficiency and allocation pressure.*
3. ***Memory streaming*** *(10^9 IoT events): Z-score anomaly detection combined with a per-sensor rolling average. Designed to contrast latency variability induced by managed garbage collection with the deterministic deallocation enforced by Rust's ownership model.*

Experimental Results

- ***GC behaviour****: G1 pauses average below 0.6 ms with a heavy tail (p99 ≈ 1 ms), confirming JVM overhead concentrates in the tail, not the typical case (see Figure 1).*
- ***Execution time****: Rust is faster across all workloads; the gap scales with I/O intensity — marginal for CPU-bound tasks (≈5 %), approaching factor of two under heavy I/O (see Figure 2).*
- ***Memory footprint:*** *Rust's peak RSS is 2.7×–50× lower, reflecting the absence of a managed runtime baseline and deterministic deallocation (see Figure 3).*

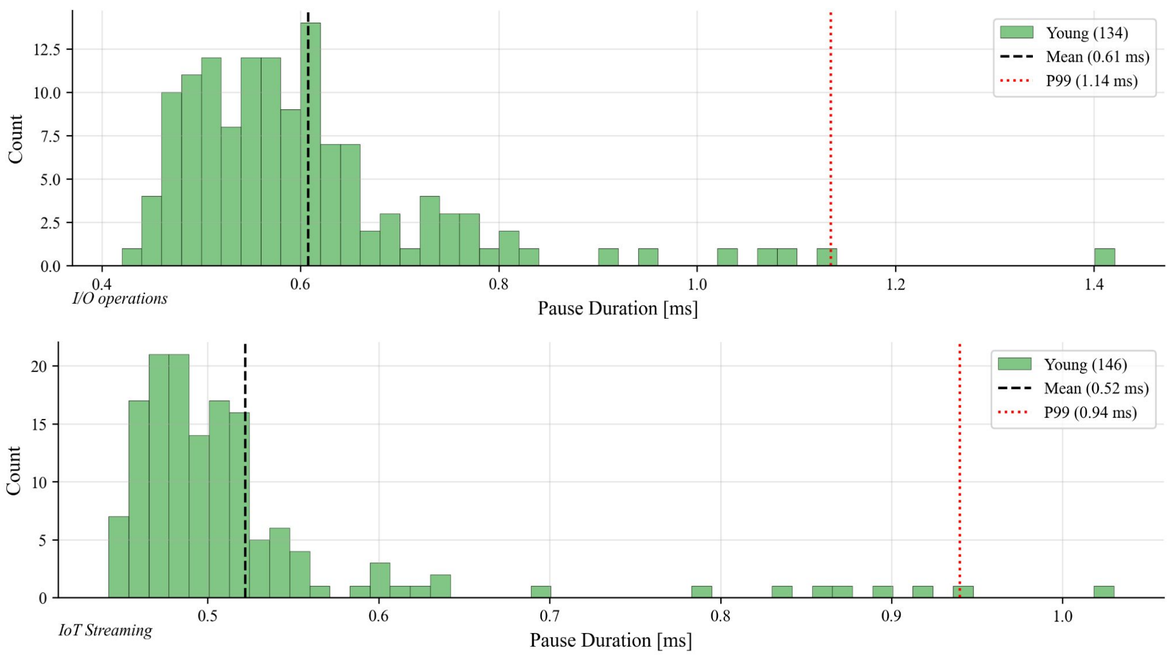


Figure 1. Garbage Collector Pauses Histogram.

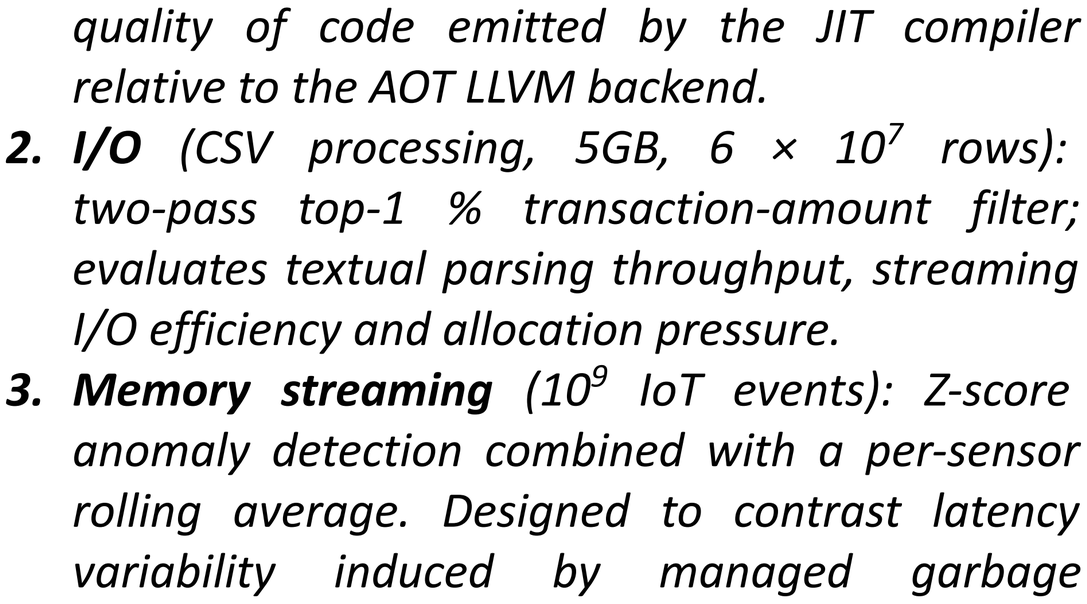


Figure 2. Execution Time.

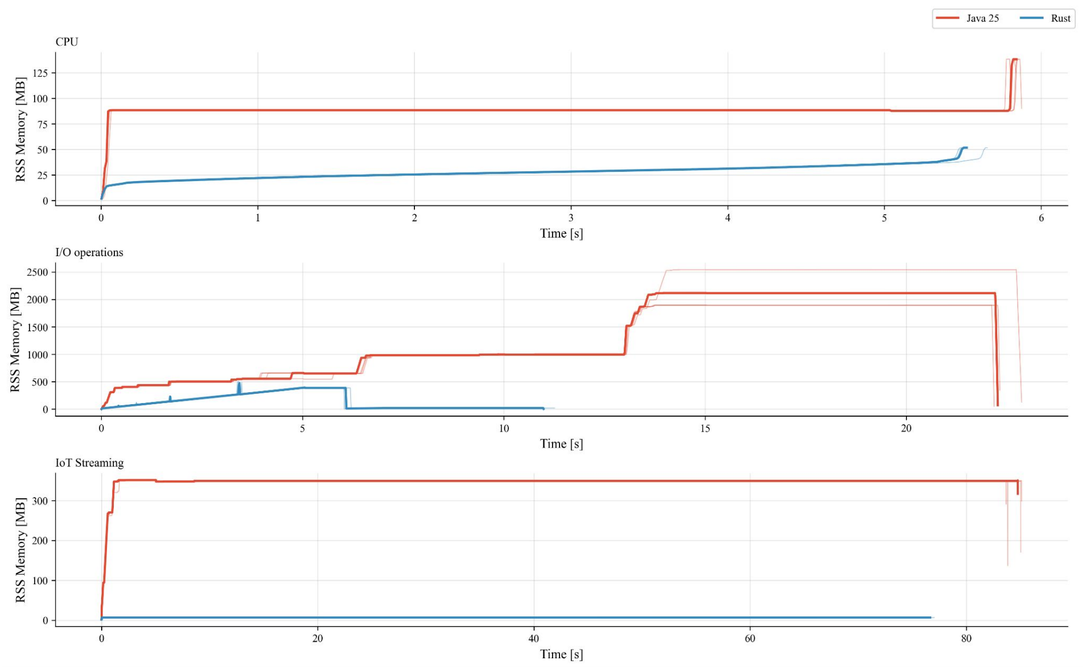


Figure 3. Memory Usage Over Time.

POLITECHNIKA LUBELSKA
LUBLIN UNIVERSITY OF TECHNOLOGY
IBS PAN
PTI POLSKIE TOWARZYSTWO INFORMATYCZNE ODDZIAŁ LUBELSKI